

I want to continue exploring the B-G431B-ESC1, I already establish a FOC (1 direction) for my Motor a 2 pair poles 2852 3100KV sensorless. But a want to change the approach to the platform, I want to achive a “RC car forward + reverse”

The control behavior i want:

Input: RC receiver PWM (typically centered around a neutral pulse; your radio endpoints vary).

Mapping: neutral  $\pm$  deadband  $\rightarrow$  command = 0

above neutral  $\rightarrow$  positive torque (forward)

below neutral  $\rightarrow$  negative torque (reverse)

Safety logic (important for drivetrains):

require neutral on startup (arming)

allow switching into reverse only when measured speed is  $\sim 0$   
(otherwise treat as braking)

apply slew-rate limiting (ramps) on torque/current to avoid breaking gears/traction loss

I want to use UART por the input command and output the best speed estimation possible. Right now i had a motor without load in where i can test things and i also had a motor attach to a car with all the reduction gear, the differentials and the wheels.

Thought for 1m 56s >

Here’s a solid way to turn your current **sensorless FOC (1 direction)** setup on the **B-G431B-ESC1** into an **RC-car style forward + brake + reverse** controller, with **UART command in** and **best-effort speed estimate out**.

## 1) Control approach that feels like an RC car ESC

For RC cars, **torque/current control** (Iq reference) usually feels right:

- Keep your inner FOC loops: **Id  $\rightarrow$  0, Iq  $\rightarrow$  torque**
- Keep your sensorless observer for speed/angle once locked
- Add a **throttle state machine** on top that decides:
  - **Drive forward** (Iq > 0)
  - **Brake** (Iq opposes current motion)
  - **Drive reverse** (Iq < 0, only after speed  $\approx 0$ )

You *can* do bidirectional in speed mode too (signed speed ref), but “brake vs reverse gating” is cleaner in torque mode.

---

## 2) Input mapping (PWM-style throttle, carried over UART)

Even if the physical receiver PWM isn’t connected to the ESC yet, you can send the *decoded pulse width* or *normalized throttle* over UART and reuse the same logic.

### Option A (recommended): UART sends normalized throttle

- UART command: thr in [-1.0 ... +1.0]
- Deadband applied around 0

### Option B: UART sends pulse width in $\mu$ s

- Typical: 1500  $\mu$ s neutral (but radios vary)

- You implement mapping:

```
float map_pwm_to_thr(int pw_us, int neutral, int deadband, int span_us)
{
    int d = pw_us - neutral;
    if (abs(d) <= deadband) return 0.0f;

    // remove deadband
    if (d > 0) d -= deadband;
    else      d += deadband;

    // ...

    return thr;
}
```

### 3) Safety + driveline logic (the key part)

You want these rules:

1. Neutral on startup (arming)
2. Reverse only if speed  $\approx 0$
3. Otherwise, negative throttle becomes **braking**
4. Slew-rate limit torque/current

## State machine (simple and robust)

States:

- DISARMED
- ARMED
- DRIVE\_FWD
- BRAKE
- DRIVE\_REV

Core thresholds (start conservative):

- `RPM_STOP` = 200 (consider "stopped" if `|rpm|` < this for a short time)
- `RPM_REV_ENABLE` = 300 (must be below this to enter reverse)
- `T_STOP_HOLD` = 200 ms (must stay below stop threshold this long)
- `Iq_max_drive` (your max drive torque current)
- `Iq_max_brake` (often same or a bit lower at first)

## Logic (pseudocode)

Run this at a fixed rate (e.g., 1 kHz torque-command task):

```
// Inputs
float thr;           // [-1..+1] from UART after deadband
float rpm_est_f;     // filtered signed mechanical rpm
float dt;

// Outputs
float Iq_cmd;        // signed torque current reference

// Config
float Iq_max_drive = 8.0f;      // example, start low and increase
float Iq_max_brake = 6.0f;      // example
float dIq_dt_limit = 50.0f;     // A/s (slew limit)
float RPM_STOP = 200.0f;
float RPM_REV_ENABLE = 300.0f;

// State
static enum {DISARMED, ARMED, DRIVE_FWD, BRAKE, DRIVE_REV} st;
static float Iq_ref = 0;
static float stop_timer = 0;

bool is_stopped = fabsf(rpm_est_f) < RPM_STOP;
```

```

switch(st)
{
case DISARMED:
    Iq_ref = 0;
    stop_timer = 0;
    if (fabsf(thr) < 0.01f) {           // neutral seen
        // require neutral for some time if you want (arming delay)
        st = ARMED;
    }
    break;

case ARMED:
    if (thr > 0.01f) st = DRIVE_FWD;
    else if (thr < -0.01f) st = BRAKE; // start with brake; reverse only after stop
    else Iq_ref = 0;
    break;

case DRIVE_FWD:
    if (thr >= 0.01f) {
        Iq_ref = thr * Iq_max_drive; // positive
    } else if (thr <= -0.01f) {
        st = BRAKE; // request reverse while moving => brake first
        stop_timer = 0;
    } else {
        Iq_ref = 0;
    }
    break;

case DRIVE_REV:
    if (thr <= -0.01f) {
        Iq_ref = thr * Iq_max_drive; // negative
    } else if (thr >= 0.01f) {
        st = BRAKE; // request forward while reversing => brake first
        stop_timer = 0;
    } else {
        Iq_ref = 0;
    }
    break;
}

// Apply braking torque opposite to current motion (or hold 0 if truly stopped)
if (is_stopped) {
    stop_timer += dt;
    Iq_ref = 0;

    if (stop_timer > 0.2f) {
        if (thr < -0.01f) st = DRIVE_REV;
        else if (thr > 0.01f) st = DRIVE_FWD;
        else st = ARMED;
    }
} else {
    stop_timer = 0;

    // Brake direction: oppose rpm sign
    float brake_dir = (rpm_est_f > 0) ? -1.0f : +1.0f;
    float brake_mag = fabsf(thr) * Iq_max_brake; // stronger brake if more negative thrott
    Iq_ref = brake_dir * brake_mag;
}
break;
}

```

## Add the torque/current slew-rate limiter (must have)

After you compute `Iq_ref`, apply a ramp limit before sending to FOC:

```

float limit_step = dIq_dt_limit * dt;
float err = Iq_ref - Iq_cmd;

Iq_cmd += err;

```

Then command `Iq_cmd` into your FOC torque mode (or whatever hook you’re using).

## 4) “Best speed estimation possible” (sensorless reality + what to do)

With **sensorless**, speed estimation is usually:

- Great at medium/high speed
- Less reliable near zero (exactly where you want to decide “allow reverse”)

So do two things:

A) Use observer speed, but output a filtered value

Convert electrical → mechanical:

- If your observer gives  $\omega_e$  (electrical rad/s) and pole pairs =  $pp$  :
  - $rpm_{mech} = \omega_e * 60 / (2\pi * pp)$

Then low-pass filter:

</> C

B) Make “stopped” detection conservative

Instead of trusting one sample:

- Require  $|rpm_f| < RPM_{STOP}$  for **200 ms**
- Also optionally require  $|Iq_{cmd}|$  small (so you’re not “holding torque”)

That’s exactly what the `stop_timer` block above does.

5) UART command + telemetry (practical and debug-friendly)

Minimal UART RX (commands)

- `T <float>\n` throttle in [-1..+1]
- `E 1|0\n` enable/disable (disarm)
- `S\n` status request

Add a **command timeout**:

- If no valid command for 300–500 ms →  $thr = 0$  (and optionally go to BRAKE or ARMED)

Telemetry TX (50–200 Hz)

Send one line (CSV) like:

`rpm_ref,rpm_est,rpm_f,thr,iq_cmd,iq_meas,vbus,temp,fault,state`

This will make tuning 10× faster.

6) Test sequence (use your no-load motor first, then car)

1. No-load

- Confirm arming requires neutral
- Confirm forward works
- While spinning forward, command negative throttle → it brakes, then only reverses after stop

2. Car on a stand (wheels off ground)

- Lower `Iq_max_drive` and `dIq_dt_limit` at first
- Verify no gear shock on transitions

3. On ground

- Increase torque gradually
- Tune:
  - deadband feel
  - braking strength curve
  - ramp limit (this protects gears)

If you tell me what you’re currently using as the base (ST MCSDK “torque mode” interface, your own FOC loop, Motor Pilot project, etc.), I can point out the exact place to insert:



- the state machine task,
- the slew limiter,
- and the telemetry packet.



I am working with ST Motor Pilot V6.4.1, STM32IDE, I don't have yet "torque mode" inteface

Thought for 5m 11s >

Got it. With **Motor Pilot + MCSDK 6.4.1**, what you’re seeing is normal: the **default Motor Pilot UI usually exposes speed (and speed ramps)**, not an “RC ESC torque throttle” widget. Motor Pilot is mainly a monitor/control tool over the **serial port using MCPV2** STMicroelectron..., and what it can command depends on what your **generated firmware/UI module** exposes.

The good news: MCSDK firmware supports **Speed or Torque control modes**, can **switch between them**, and supports **programmable speed/torque ramps** STMicroelectron.... So you have two practical paths:

### Path A (fastest): Implement RC forward/brake/reverse using Speed mode only

You can build the RC-car behavior *without torque mode* by commanding **signed speed references** and using a small state machine.

#### What you use from MC API (already available)

- `MC_ProgramSpeedRampMotor1(hFinalSpeed, hDurationms)` STMicroelectron...
- `MC_GetMecSpeedAverageMotor1()` gives average mechanical speed in **dHz** (0.1 Hz) STMicroelectron...
- `MC_GetImposedDirectionMotor1()` gives direction **+1 / -1** STMicroelectron...
- `MC_GetSpeedSensorReliabilityMotor1()` lets you gate “reverse enable” only when the estimator is **reliable** STMicroelectron...

Conversion tip (for telemetry + thresholds):

- $rpm = dHz * 6$  (because  $0.1\text{ Hz} \times 60 = 6\text{ rpm}$ ).
- Signed rpm (safe):  $rpm\_signed = direction * rpm\_mag$  using `MC_GetImposedDirectionMotor1()`.

#### RC logic (speed-mode version)

- Deadband around neutral → target speed = 0
- Forward throttle → positive speed ref
- Reverse throttle:
  - if  $|rpm| > RPM\_REV\_ENABLE$  → command speed ref = **0** (brake-to-stop)
  - only when  $|rpm| < RPM\_STOP$  for e.g. 200 ms (and reliability true) → allow negative speed ref

Important detail: MC API treats a ramp as the command primitive; “only one ramp can be programmed at a time, the last one replacing the previous” STMicroelectron.... So update your ramp at something like **50–100 Hz**, not every PWM tick.

This gets you a very ESC-like feel, especially if you:

- choose different ramp times for accel vs decel (slew limiting)
- keep reverse locked until stop (protect gears)

### Path B (better feel): Enable Torque mode and command torque ramps (Iq)

MCSDK explicitly supports **torque mode + torque ramps** STMicroelectron..., but you may not be exposing it in your current generated project/UI.

#### How to tell if your firmware already has torque control hooks

In your CubeIDE project, open `mc_api.h` and search for:

- Torque
- ControlMode
- Ramp

If you only see speed ramp APIs (like the ones in the MC API description [STM32 MC API description](#) ) and nothing torque-related, then your current configuration is effectively “speed-only” at the application API level.

What usually enables it

You typically need to enable/configure torque mode in **STM32 MC Workbench** (then regenerate) so the application layer exposes torque commands. The release notes confirm the firmware feature set includes **Speed/Torque control mode switching and torque ramps** [STM32 MC Workbench release notes](#) .

About Motor Pilot UI

Motor Pilot can connect, start the motor, and set speed via the default knob/ramp widgets [STM32 MC Workbench](#) . It can also be customized (QML) [STM32 MC Workbench](#) , but torque control won’t appear unless the firmware exposes the corresponding features/registers.

UART for command-in + speed-out (recommended wiring approach)

Because Motor Pilot uses the board’s serial link (MCPV2) [STM32 MC Workbench](#) , don’t mix your RC-throttle bytes into the same UART stream that Motor Pilot is using.

Best practice:

- Keep **one UART** for Motor Pilot (debug/monitoring/tuning)
- Use a **second UART** for your throttle command + telemetry (simple protocol)

For speed telemetry, use:

- `MC_GetMecSpeedAverageMotor1()` (dHz) [STM32 MC Workbench](#)
- and optionally include `MC_GetSpeedSensorReliabilityMotor1()` [STM32 MC Workbench](#) so your external controller knows when “near-zero” is trustworthy.

If you want the quickest next step: implement **Path A (speed-mode ESC behavior)** first (it’s enough to test your drivetrain safely). Then we switch to **Path B** once you confirm whether `mc_api.h` exposes torque-mode calls in your generated FOC project.

Tell me one thing (just copy/paste): the lines you have in `mc_api.h` for **speed ramp** (and whether you see anything that contains “Torque”). I’ll map your exact project API to the RC state machine without guessing function names.

 Sources

workspace.zip

Zip Archive

this is the project workspace so far

Thought for 23m 36s >

Perfect — I opened your `workspace.zip` and you actually **already have torque-mode at the firmware API level**, even if Motor Pilot doesn’t show a “torque knob”.

What I found in your project

1. Torque mode exists in your generated code

In `motor_test1_20260227/Inc/mc_api.h` you already have:

- `MC_ProgramTorqueRampMotor1_F(float FinalTorque_A, uint16_t ms)`
- `MC_SetCurrentReferenceMotor1_F(qd_f_t IqdRef)`
- `MC_GetControlModeMotor1()`

So the “missing torque interface” is mainly a **Motor Pilot UI limitation**, not a firmware limitation.

2. Your project currently overwrites the command using a potentiometer

In `motor_test1_20260227/Src/mc_app_hooks.c` you had:

- `SPDPOT_Run(&SpeedPotentiometer_M1, rawValue);`  
called every Medium-Frequency tick, meaning it continuously rewrites the speed reference. That must

be disabled for RC throttle control.

3. Important sensorless constraint in YOUR code

In `motor_test1_20260227/Src/mc_api.c` , the SDK warns (and you’ve likely hit this already):

**sensorless ramps cannot reverse direction**; trying to cross through 0 rpm tends to end in **Speed Feedback fault**.

Also your `drive_parameters.h` has:

- `#define OBS_MINIMUM_SPEED_RPM 6000`

So your observer is configured to be reliable only above ~6000 rpm. That means “reverse only when speed  $\approx$  0” must be implemented using **STOP → IDLE → START** logic (state machine), not by trusting the estimator near zero.

---

## What I changed for you (ready-to-test RC forward / brake-to-stop / reverse)

I created a small RC-car control layer that runs inside the **Medium Frequency Hook** and does:

- Neutral on startup (arming)
- UART throttle `T <float>` in `[-1..+1]`
- Same direction → speed ramp
- Opposite direction request while running → **brake down to safe speed**, then **STOP**, then **START in the opposite direction**
- Telemetry out over UART (rpm, rpm\_ref, mode, faults, reliability)

And I disabled the speed potentiometer logic so it won’t fight you.

### Download the modified workspace

[Download workspace\\_rc\\_modified.zip](#)

---

## What you still need to do in CubeMX / CubeIDE (1 key step)

Right now **USART2** is used by **Motor Pilot (MCP over UART)** inside `MC_RunMotorControlTasks()` . If you inject your throttle commands on USART2, you will collide with Motor Pilot traffic.

So do this:

### Step 1 — Add a second UART for RC I/O (recommended)

In the `.ioc` :

- Enable **USART3** (or USART1 / LPUART1 — any free one)
- Mode: Asynchronous, 115200 8N1
- Enable **USARTx global interrupt** (RX)
- (Optional) enable TX DMA later, but not required

Regenerate code.

### Step 2 — Call `RC_UART_Init()` once

Add this to `main.c` in `/* USER CODE BEGIN 2 */` after the UART init calls:

```
#include "rc_uart.h"

// ...

<> C
/* USER CODE END 2 */
```

That’s it — the RC control state machine is already being called from `MC_APP_PostMediumFrequencyHook_M1()` .

---

## UART command format (simple)

Send newline-terminated ASCII lines:

- T 0.30\n → forward 30%
- T -0.40\n → reverse request (will brake-to-stop then reverse)
- T 0\n → neutral (stop)
- E 0\n → disable + force stop
- E 1\n → enable (arming still requires neutral)
- S\n → request one telemetry line immediately

Telemetry (50 Hz) looks like:

```
ms=... thr=... en=... rcst=... stm=... mode=... rpm=... rpm_ref=... rel=... faults=0x....
```

---

## Notes you should be aware of (because of your current sensorless config)

- With OBS\_MINIMUM\_SPEED\_RPM = 6000 , low-speed crawling will feel “steppy” (any nonzero throttle clamps to at least ~6000 rpm). This isn’t a software bug — it’s your observer minimum speed setting.
- Your reverse logic should be based on **STOP/IDLE**, not “rpm close to zero”, because sensorless estimation is not trustworthy there (and your config reflects that).

---

## If you want true “torque throttle ESC feel” next

You already have torque APIs; once you confirm the speed-mode RC behavior is stable on the drivetrain, we can switch the “drive” part to:

- MC\_ProgramTorqueRampMotor1\_F(Iq\_A, ramp\_ms)

and keep direction change as **STOP/START** (still mandatory for sensorless reverse).

When you’re ready, tell me which UART you enabled (USART1/USART3/LPUART1) and your target battery voltage + current limit you want for the first drivetrain-safe test (example: start at 3–5 A).

     ...

Do you recommend that i follow this plan or could you recommend me to go full torque/current control?

Thought for 24s >

I’d recommend **you follow the plan you have now (speed-mode + RC state machine) for first drivetrain bring-up**, and then **move to torque/current control once the forward/brake/reverse behavior is proven safe**.

Going “full torque” immediately is doable, but with **sensorless** it’s usually where people lose time (and sometimes hardware) because low-speed estimation and direction changes are exactly the weak spots.

## Why start with speed-mode first (even if the final goal is torque)

Speed-mode gives you three big advantages right now:

### 1. Observer friendliness

Sensorless observers are most reliable above a minimum speed. Speed mode naturally keeps the motor inside a region where the estimator behaves. Torque mode can push you into low-speed/near-zero situations where angle/speed are noisy.

### 2. Safer first tests on the car

You can cap max speed easily and the loop will “self-regulate” against load changes. With torque mode, a small throttle can produce a big launch torque and shock the drivetrain if your ramps/limits aren’t dialed in.

### 3. You still get RC-car behavior

Your RC logic (arming, brake-to-stop, reverse lockout, ramps) is mostly independent of whether the “actuator” is speed ref or torque ref. So validating the state machine in speed mode is time well spent.

## Why torque/current control is better long-term (the “ESC feel”)

Once basic behavior is stable, torque/current control is the right destination because:

- **Traction feels natural:** torque maps to wheel force; speed control can hunt under load and feel “robotic.”
- **Wheel slip management is easier:** you can limit torque instead of fighting speed error.
- **Braking is controllable:** apply a defined negative torque (with limits).

**Sensorless reality: you probably want torque, but not “pure torque everywhere”**

The best practical solution for a sensorless RC drivetrain is usually **hybrid behavior**:

- **Torque control for driving and braking** once you’re in a reliable estimator region.
- **Strict STOP/START gating for direction reversal** (don’t cross through zero speed with sensorless torque control).
- **Optional speed limiter** on top of torque (keeps rpm from running away on no-load).

This gives you torque “feel” without relying on accurate near-zero estimation.

**Recommended phased plan (fast and low-risk)**

**Phase 1 — Keep your current speed-mode RC state machine (1–2 sessions)**

- Implement UART throttle + arming + brake-to-stop + reverse lockout
- Validate on the bench motor, then wheels-off-ground, then ground
- Log telemetry: commanded throttle, speed estimate, motor state, faults

Success criteria:

- No unexpected reverse engagement while moving
- Smooth ramps, no gear shock
- No recurring speed feedback faults during transitions

**Phase 2 — Switch DRIVE and BRAKE states to torque ramps (keep the same state machine)**

- In DRIVE\_FWD / DRIVE\_REV: command  $I_{q\_ref}$  proportional to throttle
- In BRAKE: command  $I_{q\_ref}$  opposite to motion, but limited
- Keep the same “reverse only after stop” rule

Add these protections before increasing torque:

- **Slew-rate limit on  $I_q$**
- **Separate limits:**  $I_{q\_max\_drive}$  and  $I_{q\_max\_brake}$  (start brake lower)
- **Bus-voltage supervision** during braking (regen can push bus voltage up)
- **Command timeout → neutral**

**Phase 3 — Optional: add a speed cap on top of torque**

- If  $rpm > rpm\_limit(throttle)$  then reduce  $I_q$  (simple clamp PI or proportional drop)
- This prevents runaway on no-load and gives you a consistent throttle feel.

**When I would *not* go full torque yet**

I would not start torque-first if any of these are true (common with new setups):

- Your current loop / current sensing isn’t fully trusted yet
- You haven’t tested braking/regen impact on your DC bus
- Your sensorless estimator has a high minimum reliable speed (common)
- You still get “speed feedback” faults during transients

**Bottom line**

- For your current stage: **follow the speed-mode plan first** to validate the RC-car state machine safely.
- After it behaves correctly: **move to torque/current control inside the same state machine** (that’s the best RC feel), but keep **STOP/START for direction changes** and add  **$I_q$  ramps + bus-voltage protection**.

If you want, I can give you the exact swap: which function calls to replace in DRIVE/BRAKE states so speed ramps become torque ramps, while keeping the rest identical.



I like the roadmap that you present, i would like to understand ahead to the implementation if we are going to continue using the motor

pilot and the workbench? or if we are going full STM32IDE ? i want to see the bigger picture of the implementation setup

Thought for 20s >

Big picture: you'll keep using **all three**, but for different roles. The end result (your "ESC firmware") will run **standalone from STM32CubeIDE**; **Motor Pilot** is **optional** and mainly for tuning/diagnostics; **Workbench** **stays the source of truth** for motor-control parameters and code generation.

The roles of each tool

1) STM32 Motor Control Workbench (MC Workbench)

Use it for:

- Motor/inverter configuration (PWM, ADC current sensing, protections, limits)
- Sensorless observer settings, startup sequence, thresholds
- Generating the MCSDK project (the "motor-control engine" + middleware glue)

Why keep it:

- It's the clean way to regenerate *consistent* MCSDK code when you change parameters.
- If you go "CubeIDE only" without Workbench, you'll end up editing generated control files manually (painful and easy to break).

In practice: **Workbench is your configuration + generator**, not your daily coding tool.

2) Motor Pilot

Use it for:

- Bringing up the motor quickly (start/stop)
- Watching variables (speed, currents, faults, bus voltage)
- Quick tuning/verification while you iterate

Why it's optional:

- Your final RC-car firmware won't need Motor Pilot at all.
- It's a debugging console. Very useful during development, unnecessary in the vehicle.

In practice: **use it during Phase 1–2**, then you can drop it or keep it as a "service port".

3) STM32CubeIDE (your "real application development")

Use it for:

- Your RC-car state machine (arming, brake-to-stop, reverse lockout)
- UART protocol for throttle + telemetry
- Safety logic (timeouts, failsafes, slew limits)
- Packaging the firmware as a standalone "ESC"

In practice: **CubeIDE is where your product behavior lives**.

Recommended implementation setup (that scales cleanly)

A) Keep generated motor-control code "as generated"

Assume Workbench may regenerate and overwrite files. So:

- Put your RC logic in **hooks / USER CODE** sections / your own `rc_*.c` files.
- Avoid editing "core generated" MCSDK files unless absolutely required.

This way you can regenerate from Workbench without losing your RC-car logic.

B) Use two UARTs during development

- UART #1: Motor Pilot / MCP (debug & monitoring)
- UART #2: Your RC throttle + telemetry (your protocol)

Reason: Motor Pilot uses a defined protocol stream; mixing your own bytes into that UART will cause conflicts.

C) Production choices (two valid options)



Option 1 (recommended): keep two ports

- One small “service/debug UART” for Motor Pilot (or a debug console)
- One “command UART” for the vehicle control

Option 2: single UART in production

- Disable Motor Pilot/MCP in the final build and reuse that UART for your commands
- Keep a compile-time switch like `#define USE_MOTOR_PILOT 0/1`

How the roadmap maps to tools

Phase 0: Stabilize motor-control baseline (Workbench + Motor Pilot)

- Ensure current sensing/FOC/stability
- Confirm sensorless startup, protections, fault behavior

Phase 1: RC behavior using speed ramps (CubelIDE + Motor Pilot)

- Implement RC state machine in CubelIDE (hooks)
- Command speed ramps (safe)
- Use Motor Pilot to watch faults and trends while testing drivetrain logic

Phase 2: Switch DRIVE/BRAKE to torque ramps (CubelIDE + Motor Pilot)

- Same RC state machine
- Replace “speed command” with “Iq/torque command”
- Keep reverse as STOP/START gating (sensorless reality)
- Use Motor Pilot to validate currents, bus voltage (regen), limits

Phase 3: Standalone ESC firmware (CubelIDE only, Motor Pilot optional)

- Final UART protocol, command timeouts, telemetry rate
- Optional: keep a debug build with Motor Pilot enabled for lab work

The mental model of the firmware

Think of it as layers:

Your RC Car App (CubelIDE)

- reads UART throttle
- runs arming + forward/brake/reverse state machine
- applies ramps/limits
- decides what to command (speed ramp now, torque ramp later)

↓ calls

MCSDK Motor Control (generated by Workbench)

- sensorless observer
- FOC current loops
- protections / fault handling
- PWM/ADC timing

↓ drives

Power stage (B-G431B-ESC1)

Telemetry is the reverse direction: MCSDK values → your UART / or Motor Pilot.

If you want, I can open your workspace structure and tell you exactly **which files you should treat as “never edit” vs “safe to edit”**, and where the RC logic should live so Workbench regeneration won’t wipe it.

Thought for 4m 36s >

Yes. Based on what’s inside your `workspace/motor_test1_20260227/` project, here’s a clean “edit policy” that will survive Workbench regenerations and gives you a scalable setup for an RC-car ESC firmware.

## 1) The three zones in your project (what they mean)

### A) Workbench/CubeMX configuration artifacts (authoritative inputs)

These files define what Workbench will regenerate:

- `motor_test1_20260227.wbdef`
- `motor_test1_20260227.ioc`
- `motor_test1_20260227.ioc.wb`
- `motor_test1_20260227.settings`

Treat these as **source-of-truth configuration**. If you want to change motor parameters, protections, sensors, MCP/Motor Pilot enable/disable, etc., you do it in Workbench and it rewrites code from these.

### B) Generated MCSDK / CubeMX code (mostly overwritten on regen)

These are the files Workbench re-creates or updates frequently:

- `Inc/` and `Src/` files like:
  - `Inc/drive_parameters.h`
  - `Inc/pmsm_motor_parameters.h`
  - `Inc/power_stage_parameters.h`
  - `Inc/parameters_conversion*.h`
  - `Inc/mc_*.h`
  - `Src/mc_*.c`
  - `Src/mc_tasks*.c`
  - `Src/mcp*.c` , `Src/aspep.c` (Motor Pilot communication)
  - `Src/stm32g4xx_*.c` (CubeMX HAL init/IT files)

### C) Your application code (you own it)

This is where your RC-car state machine, UART protocol, slew limiting, and telemetry should live:

- New files you create, e.g.
  - `Src/rc_esc/*.c`
  - `Inc/rc_esc/*.h`

This zone should be **never touched by Workbench**.

---

## 2) “Never edit” vs “safe to edit” in your workspace

### Never edit (assume overwritten or too risky)

Directories:

- `Drivers/` (HAL/CMSIS, generated/vendor)
- `MCSDK_v6.4.1-Full/` (SDK payload/reference)
- `STM32CubeIDE/` project metadata (don’t hand-edit unless you know Eclipse internals)
- `.metadata/` (workspace-local, never commit)

Files (in your `Inc/` / `Src/`):

- `Inc/drive_parameters.h`
- `Inc/pmsm_motor_parameters.h`
- `Inc/power_stage_parameters.h`
- `Inc/parameters_conversion.h` , `Inc/parameters_conversion_g4xx.h`
- `Inc/mc_config*.h` , `Inc/mc_parameters.h` , `Inc/mc_interface.h` , `Inc/mc_api.h` (read-only unless you accept regen overwrites)
- `Src/mc_config*.c` , `Src/mc_parameters.c` , `Src/mc_tasks*.c` , `Src/mc_interface.c` , `Src/mc_api.c`



- `Src/mcp*.c` , `Src/aspep.c` (unless you change MCP settings in Workbench)

Rule: if it doesn't have a USER section, assume it can be overwritten.

Safe to edit **ONLY** inside USER blocks/sections

These files in your project already contain `/* USER CODE BEGIN ... */` or `/* USER SECTION ... */` markers:

- `Src/main.c`
- `Src/stm32g4xx_it.c`
- `Src/stm32g4xx_hal_msp.c`
- `Src/stm32g4xx_mc_it.c` , `Src/stm32_mc_common_it.c`
- `Src/mc_app_hooks.c`
- `Src/mc_tasks.c` , `Src/mc_tasks_foc.c` (only in user markers)
- plus the corresponding headers: `Inc/main.h` , `Inc/mc_config.h` , `Inc/parameters_conversion*.h` , etc. (only user markers)

This is where you can safely:

- add includes
  - call your RC module
  - add debug counters
  - add UART init calls
- without fighting regeneration.

Best practice (most robust): don't rely on editing generated logic lines

Example: in your `mc_app_hooks.c` , the potentiometer speed command is outside the user section:

</> C

If you delete/comment that line directly, regeneration can bring it back.

So for anything that must change behavior (like removing the pot control), use the override method below.

3) The cleanest integration pattern (survives regeneration)

Step 1 — Create your own RC-ESC module (you own these files)

Create:

- `Inc/rc_esc/rc_esc.h`
- `Src/rc_esc/rc_esc.c`
- `Inc/rc_esc/rc_uart.h`
- `Src/rc_esc/rc_uart.c`
- `Inc/rc_esc/rc_telemetry.h`
- `Src/rc_esc/rc_telemetry.c`

These contain:

- UART RX parsing: `T <float>\n` , `E 0/1\n` , etc.
- RC state machine (arming, brake-to-stop, reverse lockout)
- ramp/slew limiter
- telemetry formatting

Step 2 — Override the weak hook (recommended)

Your `mc_app_hooks.c` functions are declared `__weak` . That's perfect: you can **override them without editing generated code**.

Create a new file you own:

- `Src/rc_esc/rc_app_hooks_override.c`

Implement:

- `void MC_APP_BootHook(void)`
- `void MC_APP_PostMediumFrequencyHook_M1(void)`

Inside these you call your module.

This completely bypasses the potentiometer logic without touching generated lines.

Minimal skeleton:

```
#include "mc_app_hooks.h"
#include "rc_esc/rc_esc.h"

void MC_APP_BootHook(void)
{
    RC_ESC_Init();
}

<> C
1
RC_ESC_Task(); // runs your state machine and issues speed/torque ramps
}
```

If you still want any original behavior from the generated hook, you can copy only the pieces you truly need (but for RC-car ESC you usually want to remove the pot and fully own this path).

Step 3 — Keep UART and peripheral init inside USER CODE in main.c

UART init calls and “one-time init” should go in main.c inside /\* USER CODE BEGIN 2 \*/ so regen keeps it.

4) Bigger-picture workflow: how you’ll work day-to-day

During development (recommended)

- Keep **Motor Pilot enabled** on USART2 (your project is already set up this way; mcp\_config.h maps it to USART2).
- Add a **second UART** for your RC command/telemetry (USART1/USART3/LPUART1).
  - This avoids mixing your throttle protocol with Motor Pilot’s MCP/ASPEP stream.

You will iterate like this:

1. Tune/control parameters in Workbench → regenerate
2. Build in CubeIDE
3. Test with Motor Pilot monitoring + your RC UART commanding

For “vehicle / standalone ESC” firmware (release)

Two options:

Option A (cleanest): disable Motor Pilot in Workbench and regenerate

- Workbench removes MCP plumbing and frees the UART.
- Your firmware becomes “pure ESC”.

Option B: keep Motor Pilot only in a debug build

- Two build configurations:
  - DEBUG\_PILOT : MCP enabled on USART2 + RC on USART3
  - RELEASE\_ESC : MCP disabled + RC uses whichever UART you want

Option A is usually easiest long-term because it avoids carrying MCP code in production.

5) Regeneration checklist (so nothing surprises you)

Every time you regenerate from Workbench:

1. Assume Inc/ and Src/mc\_\* may change.
2. Your files under Src/rc\_esc/ and Inc/rc\_esc/ should remain untouched.
3. Re-check only two integration points:
  - Your hook override file still compiles and is linked
  - Your extra UART (if configured in CubeMX) didn’t get reset by regeneration

Important note about extra UART: since your `.ioc` is Workbench-driven, regeneration may revert UART additions made in CubeMX. If that happens, we'll handle it by either:

- enabling that UART in the Workbench-generated IOC flow (if available), or
- keeping a small “post-regen” patch checklist (fast), or
- moving your RC interface to a peripheral Workbench already leaves stable.

If you want, I'll propose the exact file set and function signatures for your RC module (including speed-mode Phase 1 and torque-mode Phase 2), and I'll point to the exact MCSDK API calls in your `mc_api.h` that we should use so your module stays independent of generated internals.



before i start with this implementation, i want to ask something. Right now my implementation so far is a minimal working prototype of FOC, and you already recommended me a clean, robust path i want follow, but, which would be the result if a try to include in the mix of the FOC a EKF that allow me to estimate the speed in low motion and allow me to reverse estimate either?

Thought for 1m 22s >

Adding an EKF *can* improve the **quality of the speed estimate** near low speed, but it usually **won't remove the core sensorless limitation around zero speed** (and that's the part that matters for “reverse while crawling / near-stop”).

Why low-speed sensorless is hard (even with an EKF)

Most sensorless observers (including EKF-based ones) “see” rotor motion mainly through **back-EMF**, which is proportional to speed. When speed  $\rightarrow 0$ , back-EMF  $\rightarrow 0$ , so the system becomes **weakly observable**: the measurements don't contain enough information to uniquely determine rotor angle/direction. An EKF can still output something, but it becomes **model-driven** (prone to drift, sign ambiguity, and sudden flips if parameters are off).

So the practical result if you “mix EKF into the FOC” is:

- **Medium/high speed:** EKF can be excellent (often smoother than simple observers).
- **Very low speed / near zero:** EKF typically becomes fragile unless you add *extra excitation* (e.g., high-frequency injection) or add a sensor.

Would EKF let you reverse “reliably” while moving slowly?

It may help a bit, but not in the way you want. The risky part of reversing sensorless is not “having a speed number,” it's **having a reliable rotor angle and direction sign through zero**. Even if EKF gives you a signed speed, it can still lose the correct electrical angle at low speed, which is what triggers faults and/or torque spikes.

That's why most robust sensorless reverse implementations still do **brake-to-stop  $\rightarrow$  restart in opposite direction** (your roadmap), even when they have fancy observers.

What *does* solve low-speed / zero-speed estimation (practically)

1. **Hall sensors or encoder (best robustness, least time)**  
Your **B-G431B-ESC1** explicitly supports **Hall/encoder connections** (sensor connector J8) so you can run sensed FOC and get stable low-speed and clean reversals. DigiKey +1
2. **High-frequency injection (HFI) / saliency-based methods**  
This is the typical “true sensorless at zero speed” approach. It's also a real R&D project and depends on motor saliency (works better on IPMSM than surface PMSM; outrunners vary).
3. **EKF + injection (advanced)**  
An EKF becomes much more reliable near zero if you give it extra information via injection. But at that point you're building a research-grade estimator, not just “adding an EKF.”

Where an EKF *does* make sense in your project

If you want to explore it without derailing the ESC build:

**Best compromise:** add an EKF as an *auxiliary estimator* first.

- Run EKF at a lower rate (e.g., 1–2 kHz) using measured currents + commanded voltages.
- Use it for **telemetry** and maybe for “better stop detection” (filtered speed).
- Keep the **control** on the proven MCSDK observer + your brake-to-stop reverse gating.

This gives you learning + improved speed reporting, without making your drive unstable.

**Recommendation for your “bigger picture”**

- If your goal is a dependable RC drivetrain ESC soon: **stay with the roadmap** (speed-mode → torque-mode), keep **reverse as brake-to-stop** for sensorless.
- If you truly need **smooth creeping at low rpm and instant reverse authority**: use the board’s **Hall/encoder input** and go sensed. DigiKey +1
- If you want EKF mainly because “sensorless low speed”: EKF alone is unlikely to give you the reliability you expect at/near zero. It’s more effective when paired with injection or sensors.

If you want, I can outline a realistic “EKF add-on” plan that doesn’t break your current MCSDK control (states, signals, where to tap vd/vq and id/iq, and how to blend EKF speed with the existing estimate for UART output).

       Sources

This is the goal "your goal is a dependable RC drivetrain ESC soon: stay with the roadmap (speed-mode → torque-mode), keep reverse as brake-to-stop for sensorless."

Great — then the “bigger picture” is:

- Keep **MC Workbench** + **Motor Pilot** for parameter tuning and visibility while you iterate.
- Build your **RC ESC behavior in CubeIDE** as a thin layer on top of MCSDK.
- Implement **forward / brake-to-stop / reverse** with **STOP/START gating** (sensorless-safe).
- Move from **speed-mode** to **torque-mode** once the drivetrain behavior is proven.

Below is the clean implementation setup you can follow without getting trapped by regeneration.

**Toolchain and workflow you’ll use**

**During development (recommended)**

- 1. MC Workbench**  
Configure motor + power stage + protections + sensorless startup/observer settings → generate code.
- 2. Motor Pilot**  
Use as a “scope”: watch speed, current, bus voltage, faults while testing.
- 3. STM32CubeIDE**  
Add your RC ESC application layer (UART, state machine, ramps, failsafes). Build/flash.

**For “release firmware”**

- Either disable Motor Pilot/MCP in Workbench and regenerate a clean firmware
- Or keep it only in a debug build config

**Code ownership rules (so regeneration doesn’t hurt you)**

**You own (safe, permanent)**

Create your own module folder, for example:

- Inc/rc\_esc/\*.h
- Src/rc\_esc/\*.c

This includes:

- rc\_uart.c (throttle input protocol)
- rc\_state\_machine.c (arming + forward/brake/reverse logic)
- rc\_cmd.c (speed ramp now, torque ramp later)

- `rc_telemetry.c` (speed estimate out)

## Generated (do not edit except USER blocks)

Avoid modifying:

- `Src/mc_*.c` , `Inc/*parameters*.h` , `drive_parameters.h` , etc.

Where to “connect” your code:

- Best place: **override weak hooks** (most robust)
- Second-best: call your module from USER CODE sections in `mc_app_hooks.c` / `main.c`

## The roadmap (implementation order)

### Phase 1 — Speed-mode RC ESC behavior (fast + safe)

**Goal:** Validate logic and drivetrain safety before torque control.

What you implement:

- UART throttle in [-1..+1] with deadband
- State machine:
  - DISARMED → ARMED (requires neutral)
  - DRIVE\_FWD
  - BRAKE (if reverse requested while moving)
  - STOP/IDLE
  - DRIVE\_REV (only after stopped)
- Slew limiting using speed ramp duration (and/or update at 50–100 Hz)

What MCSDK call you use:

- `MC_ProgramSpeedRampMotor1(final_speed, duration_ms)`  
(Your existing API already has this)

Reverse gating:

- Do not try to command a negative speed while running forward.
- Use: **ramp to 0 → STOP motor → restart in opposite direction → ramp up**

Success criteria:

- No gear shock
- No “speed feedback” faults during transitions
- Repeatable arming behavior

### Phase 2 — Swap DRIVE/BRAKE to torque/current ramps (better “ESC feel”)

Keep the same state machine.

Only replace the actuator command:

- DRIVE states use:
  - `MC_ProgramTorqueRampMotor1_F(final_torque_A, duration_ms)`  
or `MC_SetCurrentReferenceMotor1_F(...)` if you want direct Iq control.
- BRAKE state applies negative torque opposite motion, but still respects stop gating.

Still keep:

- STOP/START for direction change
- command timeout → neutral
- ramp limits (dIq/dt)

### Phase 3 — Polish and “release”

- optional speed cap on top of torque
- robust failsafe (loss of UART packets)
- telemetry format finalized
- optional: remove Motor Pilot from release build

## Hardware / UART setup (important)

Keep Motor Pilot and your RC control on separate UARTs during development:

- UART for Motor Pilot (MCP/ASPEP)
- UART for RC throttle + telemetry

This avoids protocol collisions and makes debugging much easier.

## One key technical point to accept (sensorless reality)

For “dependable soon”:

- You will **not** aim for smooth creeping at near-zero in sensorless.
- You will treat reverse as: **brake to stop** → **restart reverse**.

That’s what makes it reliable.

Next step I recommend (concrete):

1. Implement Phase 1 with speed ramps.
2. Validate on bench → wheels lifted → ground.
3. Then we do the torque-mode swap.

If you want, in the next message paste (or screenshot) your current `mc_app_hooks.c` and tell me which UART you want to dedicate to RC control (USART1/USART3/LPUART1). I’ll give you the exact minimal set of files/functions to add and exactly where to connect them so Workbench regeneration won’t wipe anything.



Before I start to modify think, can you help me organice my project in git, to have a version control and branches to test ideas? which folder should i include? which ones ignore

Thought for 39s >

Yes — getting Git set up *before* you start changing things will save you a lot of pain (especially with Workbench regenerations).

Below is a practical setup for a **STM32CubeIDE + MC Workbench/MCSDK** project, including what to commit, what to ignore, and a simple branching strategy for testing ideas safely.

### 1) Where to put the Git repo (important)

Don’t make the repo at the **CubeIDE workspace root** (the folder that contains `.metadata/` ). That workspace folder is full of machine-specific Eclipse state.

Instead, create the repo at the **project folder** level, e.g.:

- Good: `workspace/motor_test1_20260227/` ← repo root
- Not recommended: `workspace/` (because it will include `.metadata/` and other noise)

If you have multiple projects you want together, you can create a “super-repo” above them, but still you must ignore `.metadata/` .

### 2) What you SHOULD commit (include in Git)

Commit everything needed to rebuild the firmware on another PC:

#### A) Workbench / CubeMX configuration (must track)

- `*.ioc`
- `*.wbdef`
- `*.ioc.wb`
- project settings files generated by Workbench (often in the project root)

These are your “source-of-truth” configs.

#### B) STM32CubeIDE project metadata (usually commit)

- `.project`
- `.cproject`
- `.mxproject` (if present)
- `.settings/` (generally ok to commit)



This makes it easy for you (and future-you) to re-open in CubelDE exactly as-is.

C) Firmware source + generated code required to build

Commit:

- Src/ , Inc/ , Core/Src , Core/Inc (whatever your project uses)
- Drivers/ , Middlewares/ (HAL/CMSIS and middleware used by build)
- startup files: startup\_\*.s , system files, linker scripts \*.ld
- MCSDK integration files that are part of the project build

Even if a lot of it is “generated,” it’s still important for reproducibility and for tracking what changed across regenerations.

3) What you SHOULD ignore (do NOT commit)

These are build artifacts, IDE caches, and machine-local junk.

Recommended .gitignore (paste this at repo root)

```
# --- Build outputs ---
**/Debug/
**/Release/
**/debug/
**/release/
**/build/
**/Build/
**/out/
**/*.o
**/*.d
**/*.a
**/*.elf
**/*.hex
**/*.bin
**/*.map
**/*.list
**/*.srec
**/*.size
**/*.su

# --- STM32CubeIDE / Eclipse workspace (do not version) ---
.metadata/
.recommenders/
RemoteSystemsTempFiles/
**/.metadata/

# --- Eclipse temp files ---
*.launch

</> gitignore

# --- VS / editor noise ---
.DS_Store
Thumbs.db
*~
*.swp
*.bak

# --- Python/Node (if you add tooling later) ---
__pycache__/
node_modules/
```



4) What about the big MCSDK\_v6.4.1-Full/ folder?

You have two good options:

Option A (recommended): do NOT commit the full SDK bundle

- Keep MCSDK installed locally (or in a separate folder outside the repo).
- Commit only your project + configs.
- Add to .gitignore if it sits next to the project:

```
</> gitignore
```



This keeps your repo lightweight.

Option B: commit the SDK (only if you need perfect reproducibility offline)

This can make the repo huge. If you choose it:

- Consider Git LFS (but it’s extra complexity).
- Or store MCSDK as a git submodule if you maintain your own copy (not always practical with ST packages).

For most development: Option A is better.

5) Branch strategy that works well for motor-control projects

Use this structure:

- `main` = always runnable / “stable”
- `dev` = integration branch (still should compile)
- feature branches for experiments:
  - `feat/rc-speed-mode`
  - `feat/rc-torque-mode`
  - `feat/uart-protocol`
  - `fix/speed-feedback-fault`

Workflow:

1. Work on a feature branch
2. Merge into `dev` when it’s working
3. Merge `dev` → `main` when you’ve tested on hardware

Tag known-good firmware states:

- `v0.1_speed_rc_ok`
- `v0.2_torque_rc_ok`

6) A commit habit that will save you later (regen vs app logic)

When you regenerate code from Workbench:

- Make one commit **only** for regeneration changes:
  - message: `regen: workbench 6.4.1 update params`
- Then a separate commit for your RC logic changes:
  - message: `feat: rc state machine + uart`

This makes diffs readable and merges much easier.

7) Minimal “getting started” commands

From inside your project folder:

```
git init
git add .gitignore
git add

</> Bash
```

Copy

```
git checkout -b dev
```

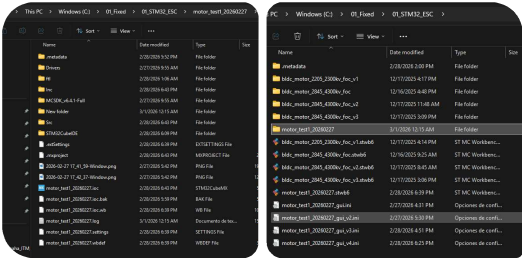
Then create your first feature branch:

```
</> Bash
```

Copy

If you tell me the exact folder layout at your repo root (just the top-level names: `Inc` , `Src` , `Drivers` , etc.), I can tailor the `.gitignore` one notch further (for example: some CubelIDE setups generate `CM4/` , `cmake-build-*` , or `*.launch` files you may want to keep or ignore).





One picture of the Workspace root and other from project inside folder

Thought for 56s >

Yes — your screenshots are perfect for deciding this.

You currently have:

- A **CubeIDE workspace root**: `C:\...\01_STM32_ESC\` that contains `.metadata/` plus multiple projects + multiple `.stwb6` files.
- A **project folder**: `...\motor_test1_20260227\` that contains `Inc/ Src/ Drivers/ ...` plus Workbench/CubeMX artifacts ( `.ioc` , `.wbdef` , etc.).

You have two good Git layouts. I’ll recommend one and give you the exact include/ignore lists for *your* folders.

## Option 1 (recommended): One Git repo for the whole workspace 01\_STM32\_ESC

Use this if you want to version-control **all projects** ( `bldc_motor_*` and `motor_test1_20260227` ) and the `.stwb6` files that live in the workspace root.

### Repo root

`C:\...\01_STM32_ESC\`

### Commit (include)

- All project folders you care about:
  - `motor_test1_20260227/`
  - `bldc_motor_*/` (whatever you want to keep)
- All Workbench project files in the root:
  - `*.stwb6`
- Workbench UI configs you want preserved:
  - `*_gui*.ini` (optional, but useful)
- Any notes/docs you add (README, wiring, tuning notes)

### Ignore

- `.metadata/` (must ignore)
- build outputs inside each project ( `Debug/ Release/` )
- big SDK bundle(s) you don’t want to track (more on that below)
- screenshots/logs/backups

## Option 2: One Git repo per project folder

Use this if you only care about `motor_test1_20260227` and you don’t want other experiments in the same history.

Repo root would be:

`C:\...\01_STM32_ESC\motor_test1_20260227\`

If you choose this, I strongly recommend you **copy** `motor_test1_20260227.stwb6` into the project folder (or a `config/` subfolder) so it gets tracked with the project.

# My recommendation for you

Go with **Option 1** right now (workspace-level repo), because:

- you already keep `.stwb6` in the workspace root,
- you clearly iterate through variants ( `*_v1` , `*_v2` , ...),
- branches will be easier than copying folders.

## A `.gitignore` tailored to your exact folders

Create this file at `01_STM32_ESC\`.gitignore :

```
# -----
# STM32CubeIDE / Eclipse workspace junk
# -----
.metadata/
.recommenders/
RemoteSystemsTempFiles/
**/.metadata/

# -----
# Build artifacts (per project)
# -----
**/Debug/
**/Release/
**/debug/
**/release/
**/build/
**/Build/
**/*.o
**/*.d
**/*.a
**/*.elf
**/*.hex
**/*.bin
**/*.map
**/*.list
**/*.srec
**/*.size
**/*.su

# -----
# Logs / backups / exports
# -----
**/*.log
**/*.bak
**/*.tmp
**/*_backup*/
**/*.zip

# -----
# Screenshots / captures (optional: keep if you want)
# -----
*.png
*.jpg
*.jpeg

</> gitignore

# ST / SDK bundles (recommended to ignore)
# -----
**/MCSDK_v6.4.1-Full/
MCSDK_v6.4.1-Full/

# If you have other SDK bundles in future:
**/MCSDK_*/

# -----
# OS/editor noise
# -----
.DS_Store
Thumbs.db
*~
*.swp
```



### Notes about a couple items

- I ignored \*.png because your workspace root contains screenshots like 2026-02-27\*.png . If you want those tracked, remove the image section.
- I ignored MCSDK\_v6.4.1-Full/ because it can make your repo heavy. Usually you commit the generated motor-control project code, not the entire SDK installer bundle.

---

## What inside motor\_test1\_20260227/ should be in Git?

Include (commit):

- Inc/
- Src/
- Drivers/
- ft1/ (keep it; it's part of your build inputs)
- .project , .cproject , .mxproject , and .settings/ directory (if present)
- Workbench/CubeMX config files already in that folder:
  - motor\_test1\_20260227.ioc
  - motor\_test1\_20260227.ioc.wb
  - motor\_test1\_20260227.wbdef
  - motor\_test1\_20260227.settings
  - .extSettings (keep; tiny and useful)

Ignore:

- Debug/ , Release/ (build outputs)
- motor\_test1\_20260227.ioc.bak
- motor\_test1\_20260227.log
- STM32CubeIDE/ folder (in your project) **unless** it contains something you explicitly rely on (in most cases it's safe to ignore; if you later see something missing, we can selectively keep it)

If you want to pre-ignore that folder too, add:

</> gitignore

---

## Recommended branch model for testing ideas safely

Use:

- main : always “runs” on hardware
- dev : integration
- feature branches:
  - feat/rc-speed-mode
  - feat/rc-uart-protocol
  - feat/rc-torque-mode
  - fix/speed-feedback

Tag known-good points:

- v0.1\_foc\_baseline\_ok
- v0.2\_rc\_speed\_ok
- v0.3\_rc\_torque\_ok

---

## Exact Git commands (PowerShell)

Run these from the workspace root 01\_STM32\_ESC :

```
cd "C:\01_Fixed\01_STM32_ESC"

git init
notepad .gitignore # paste the gitignore above, save

git add .gitignore
```

```
git add .

</> PowerShell

git branch -M main
git checkout -b dev
```

Create a feature branch:

```
</> PowerShell
```

When it works and you want to merge:

```
</> PowerShell
```

Tag a tested version:

```
</> PowerShell
```

## One habit that will save you later (Workbench regeneration)

Whenever you regenerate code:

1. Commit regeneration alone:
  - `regen: workbench parameters update`
2. Then commit your logic changes separately:
  - `feat: rc state machine`

That makes diffs readable and avoids painful merges.

If you tell me whether you want to keep the screenshots ( \*.png ) in Git or ignore them, I'll finalize the .gitignore to match your preference exactly.



git on VS Code is tracking 1244 files; it is reasonable ?

Yes — **1244 tracked files is reasonable** for a CubeIDE + MCSDK project, especially if you're committing Drivers/ + middleware + generated motor-control code.

What matters is *which* files those are. If Git is tracking a lot of junk (workspace cache + build outputs + the full MCSDK bundle), you'll feel it immediately (slow diffs, huge repo).

## What's "normal" vs "not normal"

### Normal to track (hundreds to a few thousand)

- Src/ , Inc/
- Drivers/ (HAL/CMSIS)
- motor-control generated sources
- .project , .cproject , .mxproject , .settings/
- .ioc , .wbdef , .ioc.wb , .stwb6

### Not normal to track (you should stop tracking)

- .metadata/ (workspace cache)
- Debug/ , Release/ , build/ folders
- \*.elf , \*.o , \*.map , \*.hex , etc.

- MCSDK\_v6.4.1-Full/ (the full SDK bundle) unless you intentionally want it
- \*.log , \*.bak
- big screenshot dumps (optional)

## Quick checks (PowerShell) to see what’s being tracked

Run these inside your repo:

### 1) See the biggest tracked folders

</> PowerShell

If you see .metadata , Debug , Release , Or MCSDK\_v6.4.1-Full near the top, that’s the issue.

### 2) Check if build artifacts are tracked

</> PowerShell

### 3) Check if .metadata is tracked

</> PowerShell

### 4) Check if the full MCSDK bundle is tracked

</> PowerShell

## If you find junk is tracked: fix it (safe procedure)

1. Add/confirm .gitignore rules
2. Remove the junk from Git *without deleting from disk*:

Examples:

### Remove workspace cache

</> PowerShell

### Remove build folders everywhere

</> PowerShell

### Remove the MCSDK bundle (if you didn’t want it tracked)

</> PowerShell

Then commit:

</> PowerShell

## Bottom line

- 1244 files can be perfectly fine if they’re mostly source + drivers.
- It’s **not fine** if it includes .metadata , build outputs, or the full MCSDK distribution.

If you paste the output of the “biggest tracked folders” command (the first one), I’ll tell you immediately if what you’re tracking is clean or needs trimming.



the first command does not show anything

Thought for 55s >

If that command prints nothing, it’s almost always one of these:

- 1. You ran it in **Command Prompt (cmd)**, not **PowerShell** (the `%`, `Group-Object`, etc. are PowerShell-only).
- 2. You’re **not in the repo folder**.
- 3. You have **0 tracked files** (everything is still *untracked*), so `git ls-files` returns empty.

Do these quick checks first (works in any terminal):

```
> git status
On branch master
nothing to commit, working tree clean
```

</> Bash

On Windows, if `wc` doesn’t exist, use:

In PowerShell

```
> git status
On branch master
nothing to commit, working tree clean
```

</> PowerShell

In cmd

```
> git status
On branch master
nothing to commit, working tree clean
```

</> cmd

If (git ls-files).Count is 0 but VS Code shows 1244 files

That means VS Code is showing **untracked changes**, not “tracked files”. That’s normal right after `git init`.

To see what’s driving the count (PowerShell), run:

```
> git status
On branch master
nothing to commit, working tree clean
```

</> PowerShell

If you’re in **cmd** instead, use this simpler check:

```
> git status
On branch master
nothing to commit, working tree clean
```

</> cmd



| This is the output: Count Name Group |                           |                                                                          |
|--------------------------------------|---------------------------|--------------------------------------------------------------------------|
| -----                                | -----                     |                                                                          |
| 1237                                 | motor_test1_20260227      | {motor_test1_20260227, motor_test1_20260227, motor_test1_20260227, mo... |
| 1                                    | motor_test1_20260227_g... | {motor_test1_20260227_gui_v2.ini}                                        |
| 1                                    | motor_test1_20260227_g... | {motor_test1_20260227_gui_v3.ini}                                        |
| 1                                    | motor_test1_20260227_g... | {motor_test1_20260227_gui_v4.ini}                                        |
| 1                                    | motor_test1_20260227_g... | {motor_test1_20260227_gui.ini}                                           |



```
1 01_STM32_ESC.7z      {01_STM32_ESC.7z}
1 motor_test1_20260227.s... {motor_test1_20260227.stwb6}
1 .gitignore           {.gitignore}
```

Perfect — that output is actually *good news*.

- **1237 files under** `motor_test1_20260227/` → totally reasonable for an STM32 MCSDK/CubeIDE project (Drivers + generated motor-control code adds up fast).
- The other files are just your `.ini` , `.stwb6` , `.gitignore` , and one archive.

What I would change right now:

**1) Do NOT commit `01_STM32_ESC.7z`**

It’s a big binary snapshot and will bloat your repo. Add it to `.gitignore` and remove it from Git if already staged.

Add to `.gitignore` :

</> gitignore

If it’s already staged:

</> PowerShell

**2) Make sure you are not tracking build outputs and workspace cache**

Before your first commit, run these checks:

**Are Debug/Release tracked or staged?**

</> PowerShell

If you see them, add ignores (if not already) and unstage/remove:

</> PowerShell

**Is `.metadata/` included?**

Your output doesn’t show `.metadata` , which is great. Keep it that way.

**3) Decide about `MCSDK_v6.4.1-Full/` (important)**

In your project folder screenshot, you have `MCSDK_v6.4.1-Full` **inside the project**. I recommend **ignoring it** (it’s huge and not needed if the project builds without it).

Add to `.gitignore` :

</> gitignore

Check if it’s staged:

</> PowerShell

If staged:

</> PowerShell

4) Keep these (good to track)

- motor\_test1\_20260227.stwb6 (Workbench project)
- motor\_test1\_20260227.ioc , .wbdef , .ioc.wb , .settings
- motor\_test1\_20260227\_gui\*.ini (optional but fine)

5) Your “first clean commit” sequence

From the repo root:

```
git add .gitignore
```

PowerShell

Then create branches:

```
git branch -M main
```

PowerShell

Quick sanity question (no need to answer with text, just run it)

Can you run:

PowerShell

If it shows a huge list but *no* Debug/Release and *no* MCSDK bundle and *no* .7z , you’re in a perfect place to start implementing the RC roadmap.



Now i had a working repository; everything works (builds without errors). Now i want to start adding the functionalities that we talk latter

Thought for 6m 0s >

Nice — you’re in the best place to start: **clean build + Git baseline.**

Below is a practical, regeneration-safe way to add the RC ESC features (Phase 1 = speed-mode) without breaking Motor Pilot.

0) Git: lock the baseline and branch for RC work

From the repo root:

PowerShell

I strongly recommend you commit in small steps:

1. “add rc\_esc skeleton”
2. “add UART3 + parser”
3. “add RC state machine + speed ramps”

1) Keep Motor Pilot on USART2, add a second UART for RC



Your project currently uses **USART2 for MCP/Motor Pilot** ( `mcp_config.h` shows `USARTA USART2` ). So don't mix your RC protocol on that same UART.

### Add USART3 (recommended) in the .ioc

In CubeMX (opened from the project):

- Enable **USART3** → Asynchronous
- 115200, 8N1
- Enable **USART3 global interrupt** (NVIC)
- Generate code

After generation, confirm you now have `UART_HandleTypeDef huart3;` in your project.

If later Workbench regeneration removes USART3, just re-enable it and regenerate (that's normal; we'll keep our app code isolated so it won't be lost).

## 2) Create a safe “application zone” in your project (won't be overwritten)

Create these folders/files (you own them):

Inc/rc\_esc/rc\_esc.h

Src/rc\_esc/rc\_esc.c

Inc/rc\_esc/rc\_uart.h

Src/rc\_esc/rc\_uart.c

Src/rc\_esc/rc\_app\_hooks\_override.c

### Why this layout works

- Workbench regenerates `mc_*` files.
- Your RC logic stays in `rc_esc/*` and survives regen.
- We override the weak hooks so the generated potentiometer speed control won't fight you.

## 3) Override the weak motor-control hook (so the pot control is gone)

`mc_app_hooks.c` runs the speed potentiometer before its USER section, so don't edit it. Override it.

Create: `Src/rc_esc/rc_app_hooks_override.c`

```
#include "mc_app_hooks.h"
#include "rc_esc/rc_esc.h"

void MC_APP_BootHook(void)
{
    RC_ESC_Init();
}

</> C
1
RC_ESC_FastTask(); // run state machine + issue speed ramps
}
```

This replaces the generated weak functions and stops the potentiometer logic entirely.

## 4) RC UART module (simple + robust)

Inc/rc\_esc/rc\_uart.h

```
#pragma once
#include "stm32g4xx_hal.h"
#include <stdbool.h>

void RC_UART_Init(UART_HandleTypeDef *huart);
void RC_UART_Task(void); // parse received bytes into commands
```

⌞ C



```
uint32_t RC_UART_LastCmdMs(void);           // for timeout
```

Src/rc\_esc/rc\_uart.c (IRQ RX, parse lines)

```
#include "rc_esc/rc_uart.h"
#include <string.h>
#include <stdlib.h>

static UART_HandleTypeDef *s_huart;
static uint8_t s_rx;

#define RB_SZ 256
static volatile uint8_t rb[RB_SZ];
static volatile uint16_t rb_w = 0, rb_r = 0;

static float s_thr = 0.0f;
static bool s_thr_new = false;

static bool s_en = true;
static bool s_en_new = false;

static uint32_t s_last_cmd_ms = 0;

static void rb_push(uint8_t b)
{
    uint16_t nw = (uint16_t)((rb_w + 1) % RB_SZ);
    if (nw != rb_r) { rb[rb_w] = b; rb_w = nw; } // drop on overflow
}

static bool rb_pop(uint8_t *b)
{
    if (rb_r == rb_w) return false;
    *b = rb[rb_r];
    rb_r = (uint16_t)((rb_r + 1) % RB_SZ);
    return true;
}

void RC_UART_Init(UART_HandleTypeDef *huart)
{
    s_huart = huart;
    rb_w = rb_r = 0;
    s_last_cmd_ms = HAL_GetTick();
    HAL_UART_Receive_IT(s_huart, &s_rx, 1);
}

// Call this from HAL_UART_RxCpltCallback
void RC_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart != s_huart) return;
    rb_push(s_rx);
    HAL_UART_Receive_IT(s_huart, &s_rx, 1);
}

void RC_UART_Task(void)
{
    static char line[64];
    static uint8_t idx = 0;

    uint8_t b;
    while (rb_pop(&b))
    {
        if (b == '\r') continue;

        if (b == '\n')
        {
            line[idx] = '\0';
            idx = 0;

            // Commands:
            //   T <float> (throttle -1..+1)
            //   E <0/1>   (enable)
            if (line[0] == 'T')
            {
                float v = strttof(&line[1], NULL);
                if (v > 1.0f) v = 1.0f;
                if (v < -1.0f) v = -1.0f;
            }
        }
    }
}
```

```
        s_thr = v;
        s_thr_new = true;
        s_last_cmd_ms = HAL_GetTick();
    }
    else if (line[0] == 'E')
    {
        int v = atoi(&line[1]);

</> C
        s_last_cmd_ms = HAL_GetTick();
    }
    continue;
}

    if (idx < (sizeof(line)-1)) line[idx++] = (char)b;
    else idx = 0; // line too long, reset
}
}

bool RC_UART_GetThrottle(float *thr)
{
    if (!s_thr_new) return false;
    s_thr_new = false;
    *thr = s_thr;
    return true;
}

bool RC_UART_GetEnable(bool *en)
{
    if (!s_en_new) return false;
    s_en_new = false;
    *en = s_en;
    return true;
}

uint32_t RC_UART_LastCmdMs(void) { return s_last_cmd_ms; }
```

One required glue point: HAL callback

Search your project for HAL\_UART\_RxCpltCallback . If it **doesn't exist**, add this in main.c inside a USER CODE section:

```
extern void RC_UART_RxCpltCallback(UART_HandleTypeDef *huart);

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
</> C
}
```

If it **already exists**, don't create a second one — just call RC\_UART\_RxCpltCallback(huart); inside the existing callback.

Also call init (in main.c , USER CODE BEGIN 2 after MX\_USART3\_UART\_Init() ):

```
</> C
```

5) RC ESC state machine (Phase 1 = speed-mode)

Key idea

- Use MC\_ProgramSpeedRampMotor1() with signed speed.
- For reverse: ramp to 0 → stop → restart → ramp negative.
- Add arming + timeout + deadband.

Inc/rc\_esc/rc\_esc.h

«» C



```
void RC_ESC_FastTask(void);          // called from PostMediumFrequencyHook
void RC_ESC_BackgroundTask(void);    // called from while(1) for UART parsing + telemetry
```

## Src/rc\_esc/rc\_esc.c (minimal but functional)

```
#include "rc_esc/rc_esc.h"
#include "rc_esc/rc_uart.h"
#include "mc_api.h"
#include "drive_parameters.h"    // for SPEED_LOOP_FREQUENCY_HZ
#include <math.h>
#include <stdio.h>

typedef enum {
    ESC_DISARMED = 0,
    ESC_ARMED,
    ESC_DRIVE_FWD,
    ESC_DRIVE_REV,
    ESC_BRAKE
} esc_state_t;

static esc_state_t st = ESC_DISARMED;

static float thr = 0.0f;
static bool en = true;

static uint32_t arm_ms = 0;
static uint32_t stop_hold_ms = 0;

static int16_t last_cmd_rpm = 0;

static const float DEAD = 0.05f;
static const int16_t RPM_MAX = 10000;    // start conservative, raise later
static const int16_t RPM_STOP = 200;    // "stopped" threshold
static const uint16_t RAMP_ACCEL_MS = 400; // soften driveline shock
static const uint16_t RAMP_BRAKE_MS = 250;
static const uint16_t ARM_NEUTRAL_MS = 400;
static const uint16_t STOP_HOLD_MS = 200;
static const uint32_t CMD_TIMEOUT_MS = 500;

static int16_t clamp_rpm(int32_t v)
{
    if (v > 32767) return 32767;
    if (v < -32768) return -32768;
    return (int16_t)v;
}

static int16_t get_signed_rpm_est(void)
{
    // MC_GetMecSpeedAverageMotor1() returns dHz (0.1 Hz). rpm = dHz * 6.
    int16_t dHz = MC_GetMecSpeedAverageMotor1();
    int16_t rpm_mag = (int16_t)(dHz * 6);

    int16_t dir = MC_GetImposedDirectionMotor1(); // +1 / -1
    return (int16_t)(dir * rpm_mag);
}

static float apply_deadband(float x)
{
    if (fabsf(x) <= DEAD) return 0.0f;
    // optional: rescale after deadband
    float s = (x > 0) ? 1.0f : -1.0f;
    float y = (fabsf(x) - DEAD) / (1.0f - DEAD);
    return s * y;
}

static void cmd_speed_ramp(int16_t target_rpm, uint16_t ramp_ms)
{
    if (target_rpm == last_cmd_rpm) return;
    MC_ProgramSpeedRampMotor1(target_rpm, ramp_ms);
    last_cmd_rpm = target_rpm;
}

void RC_ESC_Init(void)
{
    st = ESC_DISARMED;
    thr = 0.0f;
}
```

```

    en = true;
    arm_ms = 0;
    stop_hold_ms = 0;
    last_cmd_rpm = 0;
}

void RC_ESC_BackgroundTask(void)
{
    // parse UART commands in background (won't disturb FOC timing)
    RC_UART_Task();

    float t;
    if (RC_UART_GetThrottle(&t)) thr = t;

    bool e;
    if (RC_UART_GetEnable(&e)) en = e;

    // simple telemetry at ~20 Hz (optional)
    static uint32_t last_tx = 0;
    uint32_t now = HAL_GetTick();
    if (now - last_tx >= 50)
    {
        last_tx = now;

        int16_t rpm = get_signed_rpm_est();
        uint16_t faults = MC_GetCurrentFaultsMotor1();
        bool rel = MC_GetSpeedSensorReliabilityMotor1();

        char msg[96];
        int n = snprintf(msg, sizeof(msg),
            "st=%d thr=%.2f rpm=%d rel=%d faults=0x%04X\r\n",
            (int)st, (double)thr, (int)rpm, (int)rel, faults);

        // Non-critical blocking TX (background only)
        extern UART_HandleTypeDef huart3;
        HAL_UART_Transmit(&huart3, (uint8_t*)msg, (uint16_t)n, 10);
    }
}

void RC_ESC_FastTask(void)
{
    // fixed dt from speed loop frequency (typically 1kHz)
    const uint32_t dt_ms = (SPEED_LOOP_FREQUENCY_HZ > 0) ? (1000u / SPEED_LOOP_FREQUENCY_HZ) : 1u;

    // failsafe timeout
    uint32_t now = HAL_GetTick();
    if ((now - RC_UART_LastCmdMs()) > CMD_TIMEOUT_MS)
    {
        thr = 0.0f; // neutral on signal loss
    }

    // enable/disable logic
    if (!en)
    {
        cmd_speed_ramp(0, RAMP_BRAKE_MS);
        (void)MC_StopMotor1();
        st = ESC_DISARMED;
        arm_ms = 0;
        stop_hold_ms = 0;
        return;
    }

    float thr_db = apply_deadband(thr);
    int16_t rpm = get_signed_rpm_est();
    bool rel = MC_GetSpeedSensorReliabilityMotor1();

    bool stopped = (abs(rpm) < RPM_STOP) && rel;

    switch (st)
    {
        case ESC_DISARMED:
            cmd_speed_ramp(0, RAMP_BRAKE_MS);
            if (thr_db == 0.0f)
            {
                arm_ms += dt_ms;
                if (arm_ms >= ARM_NEUTRAL_MS) st = ESC_ARMED;
            }
            else arm_ms = 0;
            break;

        -----
    }
}

```

```

        else if (thr_db < 0.0f) { (void)MC_StartMotor1(); st = ESC_BRAKE; stop_hold_ms = 0; }
        else cmd_speed_ramp(0, RAMP_BRAKE_MS);
        break;

    case ESC_DRIVE_FWD:
        if (thr_db >= 0.0f)
        {
            int16_t ref = clamp_rpm((int32_t)(thr_db * RPM_MAX));
            cmd_speed_ramp(ref, RAMP_ACCEL_MS);
        }
        else
        {
            // reverse request while moving -> brake to stop
            st = ESC_BRAKE;
            stop_hold_ms = 0;
        }
        break;

    case ESC_DRIVE_REV:
        if (thr_db <= 0.0f)
        {
            int16_t ref = clamp_rpm((int32_t)(thr_db * RPM_MAX)); // negative
            cmd_speed_ramp(ref, RAMP_ACCEL_MS);
        }
        else
        {
            st = ESC_BRAKE;
            stop_hold_ms = 0;
        }
        break;

    case ESC_BRAKE:
        // brake-to-stop: ramp to 0
        cmd_speed_ramp(0, RAMP_BRAKE_MS);

        if (stopped)
        {
            stop_hold_ms += dt_ms;
            if (stop_hold_ms >= STOP_HOLD_MS)
            {
                (void)MC_StopMotor1(); // ensure clean stop before changing sign

                if (thr_db < 0.0f) { (void)MC_StartMotor1(); st = ESC_DRIVE_REV; }
                else if (thr_db > 0.0f) { (void)MC_StartMotor1(); st = ESC_DRIVE_FWD; }
                else st = ESC_ARMED;
            }
        }
        else stop_hold_ms = 0;
        break;
    }
}

```

### One small addition to `main.c`

Call the background task in the empty while loop (USER CODE BEGIN 3):

```

#include "rc_esc/rc_esc.h"

while (1)

</> C

}

```



This keeps UART parsing + telemetry out of the control timing.

## 6) First test procedure (Phase 1)

1. Keep Motor Pilot connected on **USART2** (unchanged).
2. Connect a USB-UART adapter to **USART3** for RC commands.
3. Send:
  - T 0.20 → should start forward smoothly
  - T -0.20 while spinning forward → should brake to zero, stop, then reverse



- `T 0 → stop`

If you get faults during transitions, we’ll tune:

- `RAMP_BRAKE_MS` , `STOP_HOLD_MS` , and thresholds
- and optionally add “wait until MC state == IDLE” using `MC_GetSTMStateMotor1()`

## 7) Commit milestones

After each step:

</> PowerShell

Then after UART works:

</> PowerShell

Then after state machine works:

</> PowerShell

If you want, paste your `main.c` UART init section (the part where `MX_USART2_UART_Init()` is called) after you enable USART3, and I’ll tell you exactly where to place `RC_UART_Init(&huart3)` and the callback so you don’t collide with any existing HAL callbacks.



The UART3 config was a wast of time, the MCU can supported by the developed Board no. Why you recommend this approach? Please forget all about this idea, and replan all the implementation that we had in mind

Updated saved memory

Thought for 1m 13s >

You’re right to push back — if your **specific B-G431B-ESC1 setup only exposes the ST-LINK Virtual COM (USART2)** and you don’t have easy access to additional UART pins/connectors, then spending time wiring/configuring USART3 is not a good use of time.

### Why I recommended UART3 in the first place

Because **Motor Pilot (MCP/ASPEP)** uses a full protocol stream on the UART. If you mix your own throttle/telemetry bytes into that same stream, *both sides* can break. So the clean engineering pattern is “one UART for Motor Pilot + one UART for your app.”

But if the board doesn’t practically give you that second UART, then the clean pattern becomes:

- **Use Motor Pilot only when you’re tuning**
- **Use your UART protocol only when you’re running the RC ESC**
- Don’t try to do both at the same time on the same serial port

Below is the replanned approach (UART3 removed completely).

## Replanned big picture (single UART: USART2 only)

### Two firmware “modes” (recommended)

You choose one of these two strategies:

**Strategy A — Two build configurations (best)**

- **DEBUG\_PILOT build:** Motor Pilot enabled on USART2 (your current workflow)
- **ESC\_RC build:** Motor Pilot disabled; USART2 is 100% yours for throttle + telemetry

This is the most robust and least confusing.

**Strategy B — One firmware, runtime switch (also good)**

Same firmware binary, but at boot it decides mode:

- If a GPIO pin is held (or a “magic” UART message arrives in first 1–2 seconds), enter **Motor Pilot mode**
- Otherwise enter **RC ESC mode**

This avoids switching builds, but requires a bit more logic.

I recommend **Strategy A** first (simpler, fewer surprises).

---

## Development workflow (what you will do in practice)

**Phase 0 — Tune FOC sensorless using Motor Pilot (keep what works now)**

- Keep your current minimal working prototype
- Use Motor Pilot to validate stability, protections, observer, speed loop settings
- When you are happy: tag it in Git ( `v0.0_foc_baseline_ok` )

**Phase 1 — RC behavior in speed mode (ESC\_RC build)**

Goal: dependable drivetrain behavior quickly.

In this build:

- Disable Motor Pilot/MCP completely
- Use USART2 for:
  - Input: throttle command (UART)
  - Output: speed estimate + faults (UART)

Implement:

- arming (neutral required)
- deadband mapping
- forward
- brake-to-stop
- reverse only after stop (STOP/START gating)
- ramps on speed reference (slew-rate limiting)

**Phase 2 — Switch DRIVE/BRAKE to torque mode (ESC\_RC build)**

Keep the same state machine; swap the command primitive:

- speed ramps → torque/current ramps (Iq)

Still keep:

- brake-to-stop reverse gating (sensorless-safe)

**Phase 3 — Harden for the car**

- UART timeout failsafe
- fault handling behavior (what to do on speed feedback / overcurrent / etc.)
- limits and ramp tuning for gears/traction

---

## How to disable Motor Pilot cleanly (high level)

You want the ESC\_RC build to **not compile/run MCP/ASPEP tasks**.

Typical MCSDK projects gate this with compile-time options/macros (and/or Workbench settings). The cleanest way is:

1. **In MC Workbench:** disable “Motor Pilot / Communication interface” (MCP over UART) for the ESC build configuration, then regenerate.



- 2. Or (if you keep one generated config): create a build flag like `USE_MOTOR_PILOT=0/1` that excludes MCP source or prevents MCP init/task from running.

Result: USART2 becomes your dedicated RC serial interface.

If you tell me which path you prefer (A: two builds, B: runtime switch), I'll give you the exact concrete steps and file locations to touch in your project.

---

## What USART2 will carry in RC ESC mode

Keep it simple and robust. Example ASCII protocol:

- `T 0.25\n` → 25% forward
- `T -0.40\n` → request reverse (brake-to-stop then reverse)
- `E 0\n` / `E 1\n` → disable/enable
- Telemetry at 20–50 Hz:  
`rpm=..., rpm_ref=..., st=..., faults=0x....`

This works great over ST-LINK VCP.

---

## What changes in your code structure (still regeneration-safe)

This part stays the same as before (UART3 removed, but the architecture remains):

- Put your RC ESC logic in your own folder:
  - `Src/rc_esc/*`
  - `Inc/rc_esc/*`
- Connect it via:
  - overriding weak hooks ( `MC_APP_PostMediumFrequencyHook_M1` , etc.), or
  - calling from USER CODE blocks

So Workbench regenerations don't wipe your RC logic.

---

## Immediate next step (so we don't waste more time)

Tell me which strategy you want:

- 1. Two build configurations (DEBUG\_PILOT vs ESC\_RC), or
- 2. One firmware with runtime mode selection

I'll then give you a step-by-step plan for:

- how to disable Motor Pilot safely,
- how to reuse USART2 for your RC protocol,
- and exactly where to insert the RC state machine for Phase 1 (speed mode).



I think this should be: create a build flag like `USE_MOTOR_PILOT=0/1` that excludes MCP source or prevents MCP init/task from running.

Thought for 3m 51s >

Perfect — that's the right approach for your board constraints: **one physical UART (USART2)**, and a **build-time switch** that decides whether that UART is owned by **Motor Pilot (MCP/ASPEP)** or by **your RC-ESC protocol**.

Here's the clean plan, with the exact places you must gate in your current MCSDK project.

---

## Goal

- `USE_MOTOR_PILOT = 1`  
Motor Pilot works normally (ASPEP + MCP on USART2 at the high baudrate you already have).

- `USE_MOTOR_PILOT = 0`  
Motor Pilot code is **not started** and its ISR/DMA logic is **not executed**.  
USART2 becomes a normal UART for **throttle input + telemetry output**.

No UART3, no extra wiring.

## Step 1 — Create 2 build configurations in STM32CubeIDE

1. Project → Build Configurations → Manage...
2. Create two configs (copy from your current working one):
  - PILOT
  - RC\_ESC

Then set preprocessor symbols:

### PILOT config

Project → Properties → C/C++ Build → Settings → MCU GCC Compiler → Preprocessor

- Add:
  - `USE_MOTOR_PILOT=1`

### RC\_ESC config

Add:

- `USE_MOTOR_PILOT=0`
- `RC_UART_BAUD=115200` (or keep 1843200 if you prefer; 115200 is just easier)

## Step 2 — Patch the 3 “ownership points” in the generated code

These are the three places where Motor Pilot takes control of USART2:

### (A) Src/mc\_tasks.c — starts ASPEP + runs MCP packet handling

1. In `MCboot()` , you currently have:

```
</> C
```

```
FOC_Init();

</> C
#endif
```

2. In `MC_RunMotorControlTasks()` , wrap the MCP processing block:

Find this block (it starts with `MCP_Over_UartA.rxBuffer = ...` and ends before `hMFTaskCounterM1 = ...` ), and wrap it:

```
#if USE_MOTOR_PILOT
    MCP_Over_UartA.rxBuffer = MCP_Over_UartA.pTransportLayer->FRXPacketProcess(...);

</> C
```

That prevents Motor Pilot from consuming/producing bytes in RC mode.

### (B) Src/stm32\_mc\_common\_it.c — USART2 IRQ handler is currently “owned” by ASPEP

This file is critical: right now it defines `USART2_IRQHandler` using LL/DMA for ASPEP.

1. Guard the include at the top:

Change:

```
</> C
```

To:

```
... #include <...>

</> C
```

2. Replace the USART2 IRQ body with a compile-time split:

```
void USART2_IRQHandler(void)
{
    #if USE_MOTOR_PILOT

        // keep the existing ASPEP/MCP handler body exactly as-is

    #else

</> C

    #endif
}
```

This is what makes USART2 usable with normal HAL RX callbacks in RC mode.

3. In SysTick\_Handler() , there is a DMA polling section for ASPEP RX:

```
if (LL_DMA_IsActiveFlag_TC(DMA_RX_A, DMACH_RX_A))
{
</> C
}
```

Wrap it:

```
#if USE_MOTOR_PILOT
    if (LL_DMA_IsActiveFlag_TC(DMA_RX_A, DMACH_RX_A))
    {
        ...
    }
</> C
,
#endif
```

Keep MC\_RunMotorControlTasks(); always running.

(C) Src/main.c — set a friendly baudrate for RC mode (optional but recommended)

Your USART2 init is currently:

- BaudRate = 1843200 (Motor Pilot-friendly)

In MX\_USART2\_UART\_Init() , inside:

```
...
</> C
...
```

Add:

```
#if !USE_MOTOR_PILOT
    huart2.Init.BaudRate = RC_UART_BAUD;

</> C
#endif
```

This way:

- PILOT build keeps 1843200
- RC\_ESC build switches to 115200 automatically (or whatever you define)

### Step 3 — RC UART input + telemetry (USART2 only)

In RC\_ESC build, you now use USART2 with HAL receive interrupts:

1. In `main.c` after `MX_USART2_UART_Init();` (USER CODE section), call your init:

```
...
// ...

</> C
```

2. Implement `HAL_UART_RxCpltCallback()` (or add to an existing one) and feed bytes to your RC parser (only in RC mode).

This is where your throttle protocol comes in.

### Why this works

- In PILOT build, you don’t change behavior: ASPEP starts, MCP runs, USART2 IRQ is the ASPEP one.
- In RC\_ESC build:
  - ASPEP never starts
  - MCP packet processing never runs
  - USART2 IRQ becomes a normal HAL UART IRQ
  - SysTick stops poking ASPEP DMA flags
  - USART2 belongs to your RC ESC

### One practical note (regen from Workbench)

Workbench regeneration can overwrite `mc_tasks.c` and `stm32_mc_common_it.c` . With Git this is manageable:

- Keep your changes as **one small commit**:

```
chore: add USE_MOTOR_PILOT gating for USART2
```
- After regeneration, if those files revert, you can re-apply quickly (or use `git cherry-pick` / `git apply` ).

If you want, paste your current `USART2_IRQHandler` section (or just confirm you’re using the same `stm32_mc_common_it.c` as in your workspace zip), and I’ll give you the exact “diff-style” patch blocks so you can copy/paste without hunting the right lines.



You’re currently looking at the **standard Eclipse build configs**: only **Debug** and **Release**. That’s normal. We’ll create **two more** (or rename/copy) so you can switch between **Motor Pilot** and **RC ESC** with one click.

### Step 1: Create two new configurations

In that “Manage Configurations” window:

1. Click **New...**
2. Choose:
  - **Copy settings from:** `Debug`
  - Name it: `Debug_PILOT`
3. Click **OK**

Repeat:

1. **New...**
2. Copy from `Debug`
3. Name it: `Debug_RC_ESC`
4. **OK**

(You can later do the same for Release if you want: `Release_PILOT` , `Release_RC_ESC` .)

Now you’ll have 4 entries:

- `Debug` (original)
- `Release` (original)
- `Debug_PILOT`
- `Debug_RC_ESC`

Select **Debug\_PILOT** and click **Set Active** when you want Motor Pilot. Select **Debug\_RC\_ESC** when you want your UART throttle firmware.

---

### Step 2: Add the build flag in each configuration

For each of the two new configs, set the preprocessor symbol:

**In CubeIDE:**

**Project → Properties → C/C++ Build → Settings → MCU GCC Compiler → Preprocessor**

There’s a field called **Defined symbols (-D)**.

**For** `Debug_PILOT`

Add:

- `USE_MOTOR_PILOT=1`

**For** `Debug_RC_ESC`

Add:

- `USE_MOTOR_PILOT=0`
- (optional) `RC_UART_BAUD=115200`

Apply/OK.

Tip: make sure the top dropdown (if shown) is set to the configuration you’re editing (`Debug_PILOT` vs `Debug_RC_ESC`). CubeIDE sometimes edits only the active config if you’re not careful.

---

### Step 3: Verify the flag is really active

In each configuration, do:

- **Project → Clean**
- **Build**

Then open **Console** build output and look for `-DUSE_MOTOR_PILOT=...` in the gcc command line.

Or simpler: add a quick compile-time check in any `.c` file temporarily:

```
#if USE_MOTOR_PILOT
#warning "Building with Motor Pilot ENABLED"

</> C

#endif
```

Build each config and confirm the warning changes.

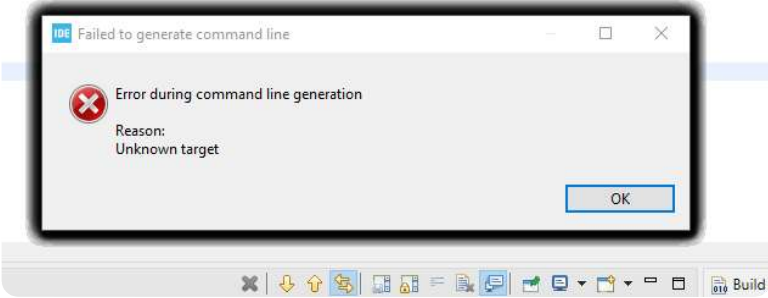
### Step 4: Next (after this) — gate the Motor Pilot code paths

Once you confirm the two configs build with different `USE_MOTOR_PILOT` values, we'll do the 3 code gates:

- 1. `mc_tasks.c` : `ASPEP_start()` and MCP packet handling
- 2. `stm32_mc_common_it.c` : `USART2_IRQHandler` (ASPEP vs HAL) + SysTick DMA poke
- 3. `main.c` : optionally set a friendlier USART2 baud in RC mode

But don't change code yet until the configs are created and verified.

If you tell me whether you successfully created `Debug_PILOT` and `Debug_RC_ESC` , I'll give you the exact copy/paste blocks for the 3 gating edits in your specific project files (with the right function names and minimal line changes).



That error (“**Failed to generate command line → Unknown target**”) usually means CubeIDE’s managed build can’t map the selected configuration to a valid build target. In STM32CubeIDE this most often happens when:

- the new configuration was created, but the **build artifact type** (ELF target) wasn’t copied correctly, or
- the project has a **misconfigured toolchain/target name** for that configuration, or
- the configuration exists but is not tied to the MCU/GCC build settings (common after importing projects or when the project uses an unusual build setup).

Let’s fix it in the most reliable way (no guessing, minimal clicks).

### Fix path that works most of the time

#### 1) Make sure you’re editing the correct project type

Right-click the project → **Properties** → confirm you see:

- **C/C++ Build**
- **MCU GCC Compiler**

If those pages aren’t there, the project isn’t seen as a managed MCU GCC project and configs won’t work.

#### 2) Recreate configs using the “Build Configurations” menu (not only Manage)

Do this:

- 1. Right-click project → **Build Configurations** → **Manage...**
- 2. Delete the broken new configs you created (keep only original Debug/Release for now).
- 3. Now use:
  - Right-click project → **Build Configurations** → **Duplicate Active**



- Name it `Debug_PILOT`
- Duplicate again → `Debug_RC_ESC`

This “Duplicate Active” path copies the actual build target metadata more reliably than some “New...” flows.

### 3) Set the active configuration and regenerate build artifacts

- Right-click project → **Build Configurations** → **Set Active** → `Debug_PILOT`
- Then: Project → **Clean...** → Clean all
- Then: **Build**

Repeat for `Debug_RC_ESC` .

If it still fails, continue below.

---

## If it still says “Unknown target” (next strongest fix)

### 4) Reset the build command for that configuration

Right-click project → **Properties** → **C/C++ Build**

- Make sure **Use default build command** is enabled
- Build command should typically be something like `make` (CubeIDE manages it)
- In the **Builder Settings**, click **Restore Defaults** (if available)
- Apply/OK, then Clean + Build again

### 5) Reindex the configuration’s “Artifact name / type”

Go to:

Project → Properties → **C/C++ Build** → **Settings** → **MCU GCC Linker**

- Check:
    - **Artifact name** is not empty (often `${ProjName}` )
    - **Artifact extension** is `elf`
- If this is blank for the new config, that can trigger the error.

Apply, then build.

---

## The “always works” fallback if configs keep breaking

If CubeIDE won’t allow multiple configs cleanly, we can still achieve the same result with **one config** by toggling a macro in a single header file:

- Create `Inc/app_build_config.h`
- Put:

```
</> C
```

- Include it wherever needed.

Not as elegant as multi-config, but it’s robust and fast.

---

## Quick question-free check: show me one thing from your project

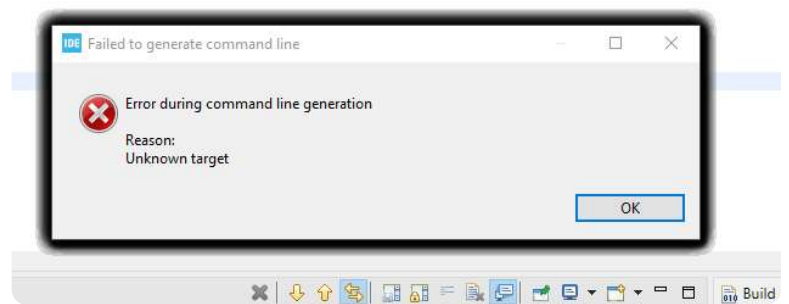
Please do this and paste the result:

Right-click project → **Properties** → **C/C++ Build** → **Settings** and tell me:

- Under **Tool Settings**, do you see “**MCU GCC Compiler / Linker**”?

If yes, we’ll keep pushing the multi-config fix.

If no, your project is not set up as “managed MCU GCC build” in the way CubeIDE expects, and I’ll give you the correct repair steps (it’s usually an import/settings issue).



this warning show up

Thought for 27s

That popup is coming from CubelIDE’s build system when it tries to use a **new configuration** whose toolchain/target metadata didn’t get created correctly. Your original **Debug/Release** configs build fine, so the fastest, cleanest way forward is:

Use the existing two configs as your two “modes”

- **Debug** = **PILOT** (Motor Pilot owns USART2)
- **Release** = **RC\_ESC** (your UART protocol owns USART2)

This gives you the `USE_MOTOR_PILOT=0/1` behavior you want, without fighting CubelIDE’s “Unknown target” bug.

Step 1) Delete the broken custom configurations

Project → Build Configurations → Manage...

Remove the new configs that trigger “Unknown target”, keep only:

- Debug
- Release

Step 2) Define the flags per configuration

Right-click project → Properties → C/C++ Build → Settings → **MCU GCC Compiler** → **Preprocessor**

Set:

Debug (PILOT)

Add defined symbol:

- `USE_MOTOR_PILOT=1`

Keep your existing USART2 baudrate for Motor Pilot (often 1843200).

Release (RC\_ESC)

Add defined symbols:

- `USE_MOTOR_PILOT=0`
- `RC_UART_BAUD=115200` (or whatever you want)

Apply/OK.

Step 3) Make USART2 baud switch automatically in code

In `main.c` , inside a USER CODE section after `MX_USART2_UART_Init();` (or inside `MX_USART2_UART_Init()` USER block), add:

```
#if !USE_MOTOR_PILOT
    huart2.Init.BaudRate = RC_UART_BAUD;
</> C
#endif
```

So:

- Debug build keeps Motor Pilot baud
- Release build uses a friendly RC baud



Step 4) Gate Motor Pilot runtime ownership (the critical part)

Now we implement the `#if USE_MOTOR_PILOT` guards in these files:

1. `mc_tasks.c`
- wrap `ASPEP_start(&aspepOverUartA);`
  - wrap the whole MCP packet handling block in `MC_RunMotorControlTasks()`
2. `stm32_mc_common_it.c`
- in `USART2_IRQHandler()` :
    - if `USE_MOTOR_PILOT==1` : keep ASPEP handler
    - else: call `HAL_UART_IRQHandler(&huart2);`
  - in `SysTick_Handler()` : wrap the ASPEP DMA “data received” section

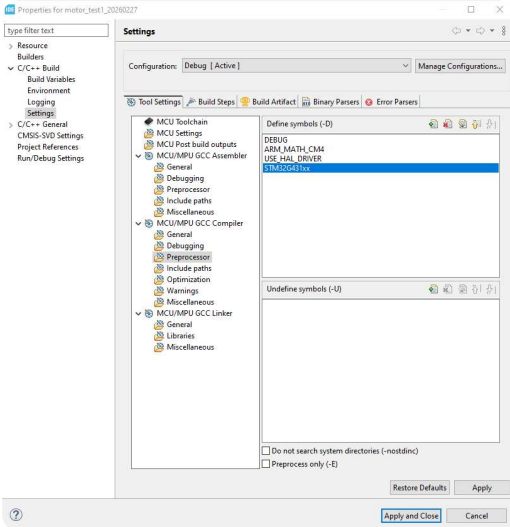
That makes USART2 fully yours in Release.

Why this is better right now

- You already have two configs that CubeIDE understands (no “Unknown target”).
- Switching mode becomes: **Build Debug for Pilot** / **Build Release for RC**.
- Later, if you still want `Debug_PILOT` and `Debug_RC_ESC` , we can revisit it; but you don’t need it to progress.

Next actionable step

Do the “Debug=1 / Release=0” preprocessor setup first. Once that’s done, tell me “flags set”, and I’ll give you the exact copy/paste `#if USE_MOTOR_PILOT` patches for `mc_tasks.c` and `stm32_mc_common_it.c` based on the code structure in your workspace.



Thought for 8s

Great — you’re in the exact right place. That panel is where we’ll set the per-configuration macro.

Set `USE_MOTOR_PILOT` for Debug (Motor Pilot mode)

You’re currently on **Configuration: Debug [Active]** (top dropdown).

1. In the left tree, stay at:  
MCU/MPU GCC Compiler → Preprocessor
2. In **Define symbols (-D)** (the list you see), click the **green “+”** button.
3. Add this symbol exactly:
  - `USE_MOTOR_PILOT=1`
4. Click **Apply** (don’t close yet).

Set `USE_MOTOR_PILOT` for Release (RC ESC mode)

Now change the configuration at the top:

1. Top dropdown **Configuration**: change from `Debug [Active]` to **Release**
2. Same place: **MCU/MPU GCC Compiler** → **Preprocessor**

3. Add:
- USE\_MOTOR\_PILOT=0
4. Also add (recommended):
- RC\_UART\_BAUD=115200
5. Click **Apply and Close**

Quick verification

1. Build **Debug** → should still work with Motor Pilot.
2. Build **Release** → should still build (same code, different macro).

If you want a super clear proof, temporarily add this to any `.c` file (then remove it later):

```
#if USE_MOTOR_PILOT
#warning "PILOT build: Motor Pilot enabled"

</> C

#endif
```

Build Debug and Release and you’ll see the warning change.

Once you confirm those two macros are set (Debug=1, Release=0), we’ll do the next step: **wrap the MCP/ASPEP ownership of USART2** in `mc_tasks.c` and `stm32_mc_common_it.c` so that in Release the USART2 interrupt is handled by HAL for your RC protocol.



where i should put the code? remember i need the more step-by-step process

Thought for 52s >

Got it. Since you’re using **USART2 only** and you want a **build flag** to switch ownership (Motor Pilot vs RC protocol), here is exactly **where** each piece of code goes, in a step-by-step order. Do each step and then build the corresponding configuration to confirm nothing broke.

Step 1 — Confirm the flags exist (no code edits yet)

You already know where to set them (Project Properties → Preprocessor). Make sure:

- Debug:** USE\_MOTOR\_PILOT=1
  - Release:** USE\_MOTOR\_PILOT=0 and (optional) RC\_UART\_BAUD=115200

Now build **Debug** and **Release** once to confirm both still compile.

Step 2 — Make Motor Pilot “do nothing” in RC build

This code goes in:

**File:** motor\_test1\_20260227/Src/mc\_tasks.c

**2.1 Wrap** `ASPEP_start(...)` **in** `MCboot()`  
Search for `MCboot(` and inside it find this line:

```
</> C
```

Replace with:

```
</> C
#endif
```

2.2 Wrap the MCP packet-processing block in MC\_RunMotorControlTasks()

In the same file, search for MC\_RunMotorControlTasks( and look for the block that starts with something like:

```
</> C
```

and ends with something like:

```
</> C
```

Wrap that whole block like this:

```
#if USE_MOTOR_PILOT
</> C
#endif
```

Now build:

- Build **Debug** (should behave as before)
- Build **Release** (should still compile)

At this point, Release no longer runs Motor Pilot logic, but **USART2 interrupt may still be owned by ASPEP**. Next step fixes that.

Step 3 — Give USART2 interrupt back to HAL in RC build

You must edit the file that defines USART2\_IRQHandler .

3.1 Find where USART2\_IRQHandler is

In CubelIDE: press **Ctrl+Shift+R** and search: USART2\_IRQHandler

You will likely find it in:

- motor\_test1\_20260227/Src/stm32\_mc\_common\_it.c

(If it’s in stm32g4xx\_it.c instead, apply the same idea there.)

**File:** motor\_test1\_20260227/Src/stm32\_mc\_common\_it.c

3.2 Guard mcp\_config.h include

At the top, find:

```
</> C
```

Change to:

```
#ifndef MOTOR_PILOT
</> C
```

3.3 Replace the body of USART2\_IRQHandler

Find:

```
void USART2_IRQHandler(void)
</> C
```

```
    // ASPEP / LL handler stuff...
}
```

Change it to this pattern:

```
void USART2_IRQHandler(void)
{
    #if USE_MOTOR_PILOT

        // Keep the existing ASPEP/MCP handler body EXACTLY as it currently is.

    #else

</> C

    #endif
}
```

### 3.4 Guard the ASPEP DMA “poke” in SysTick\_Handler

In the same file, find `SysTick_Handler(void)` and look for the part that checks DMA flags and calls:

```
</> C
```

Wrap only that DMA/ASPEP block:

```
-----
</> C
-----
```

Now build again:

- **Debug:** should still compile and Motor Pilot should still connect
- **Release:** should compile and now USART2 is HAL-owned

This step is the key “UART ownership switch”.

## Step 4 — Put your RC UART code in your own files (safe from regeneration)

Create your own folder/files (you own them, Workbench won’t overwrite):

- `motor_test1_20260227/Inc/rc_esc/rc_uart.h`
- `motor_test1_20260227 Src/rc_esc/rc_uart.c`

Start with the smallest possible test: “receive a line and reply OK”.

`Inc/rc_esc/rc_uart.h`

```
#pragma once
#include "stm32g4xx_hal.h"

</> C

void RC_UART_RxCpltCallback(UART_HandleTypeDef *huart);
```

`Src/rc_esc/rc_uart.c`

```
#include "rc_esc/rc_uart.h"
#include <string.h>
#include <stdio.h>

static UART_HandleTypeDef *s_huart = 0;
static uint8_t s_rx = 0;

static char line[64];
```

```
static uint8_t idx = 0;

void RC_UART_Init(UART_HandleTypeDef *huart)
{
    s_huart = huart;
    idx = 0;
    HAL_UART_Receive_IT(s_huart, &s_rx, 1);
}

void RC_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart != s_huart) return;

    char c = (char)s_rx;

    if (c == '\r') { /* ignore */ }
    else if (c == '\n')
    {
        line[idx] = '\0';
        idx = 0;

        // Minimal test command: "PING"
        if (strcmp(line, "PING") == 0)
        {
            const char *msg = "PONG\r\n";
            HAL_UART_Transmit(s_huart, (uint8_t*)msg, (uint16_t)strlen(msg), 20);
        }
        else
        {
            const char *msg = "OK\r\n";
            HAL_UART_Transmit(s_huart, (uint8_t*)msg, (uint16_t)strlen(msg), 20);
        }
    }

    HAL_UART_Receive_IT(s_huart, &s_rx, 1);
}

void RC_UART_Task(void)
{
    // Keep empty for now (we'll add throttle parsing later)
}
```



## Step 5 — Call your RC UART init in main.c (Release only)

File: motor\_test1\_20260227/Src/main.c

1. Add include near the top (inside a USER CODE include section if present):

```
</> C
```



2. After MX\_USART2\_UART\_Init(); in main() add (inside /\* USER CODE BEGIN 2 \*/ block):

```
#if !USE_MOTOR_PILOT
    // Optional: switch baud for RC mode
    #ifdef RC_UART_BAUD

</> C
    HAL_UART_Init(&huart2);
    #endif

    RC_UART_Init(&huart2);
#endif
```



3. Add the HAL callback (only if you don't already have one). Put it in main.c inside a USER CODE section:

```
</> C
```



```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    #if !USE_MOTOR_PILOT
        RC_UART_RxCpltCallback(huart);
    #endif
}
```

Now build **Release**, flash, open a serial monitor on USART2:

- Send `PING` + enter → you should get `PONG`

If that works, you now have a clean foundation: Release owns USART2 for RC.

---

## Step 6 — Only after PING/PONG works: add the RC state machine

Next we'll add:

- throttle command `T <float>`
- arming + deadband
- speed ramps and brake-to-stop reverse gating

But don't start that until Step 5 is confirmed.

---

If you tell me where your `USART2_IRQHandler` was found (which file), I can give you an exact copy/paste version of Step 3 that matches your current handler contents, so you don't risk breaking the Motor Pilot side in Debug.

